



青岛科技大学
Qingdao University of Science & Technology

数据结构与算法分析2

第3章 穷举算法1

梁宏涛

lht@qust.edu.cn

周二&周四上午

1/129

提纲
CONTENTS

第3章—穷举法

3.1 穷举法概述

3.2 算法优化中常用的数据结构

3.3 回文串个数

3.4 最大连续子序列和

3.5 求幂集

3.6 0/1背包问题

3.7 求全排列

3.8 n皇后问题

3.9 任务分配问题

3.10 旅行商问题

2/122

提纲
CONTENTS

第3章—穷举法

3.1 穷举法概述

3.2 算法优化中常用的数据结构

3.3 回文串个数

3.4 最大连续子序列和

3.5 求幂集

3.6 0/1背包问题

3.7 求全排列

3.8 n皇后问题

3.9 任务分配问题

3.10 旅行商问题

3/122

3.1 穷举法概述

3.1.1 什么是穷举法

- 穷举法又称枚举法或者列举法，其基本思想是先确定有哪些穷举对象和穷举对象的顺序。
- 按穷举对象的顺序逐一列举每个穷举对象的所有情况，再根据问题提出的约束条件检验哪些是问题的解，哪些应予排除。
- 主要用于解决“是否存在”和“有多少可能性”等类型的问题。

4/122

常用的列举方法

- **顺序列举**。是指答案范围内的各种情况很容易与自然数对应甚至就是自然数，可以按自然数的变化顺序去列举。
- **组合列举**。当答案的数据形式为一些元素的组合时，往往需要用组合列举。组合是无序的。
- **排列列举**。有时答案的数据形式是一组数的排列，列举出所有答案所在范围内的排列，为排列列举。

穷举法优点

- ① 理论上讲穷举法可以解决可计算领域中的各种问题。
- ② 在实际应用中通常用于解决问题规模不大的问题。
- ③ 穷举法算法一般逻辑清晰，编写的程序简洁明了。
- ④ 穷举法算法一般不需要特别证明算法的正确性。
- ⑤ 穷举法可作为某类问题时间性能的底限，用来衡量同样问题的更高效率的算法。

3.1.2 穷举算法的框架

假设问题的枚举变量是 x 和 y ，穷举次序是先 x 后 y ，均为顺序列举方式，它们的取值范围即值域分别是 $x \in (x_1, x_2, \dots, x_n)$ ， $y \in (y_1, y_2, \dots, y_m)$ ，约束条件为 $p(x_i, y_j)$ 。

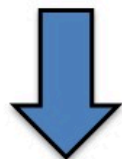
```
void exhaustive(x,n,y,m) {  
    for (int i=1;i<=n;i++) {  
        for (int j=1;j<=m;j++) {  
            ...  
            if (p(x[i],y[j]))  
                输出一个解;  
            ...  
        }  
    }  
}
```

//穷举算法框架
//枚举 x 的所有可能的值
//枚举 y 的所有可能的值
//检测是否满足约束条件

7/122

【例3.1】在象棋算式里，不同的棋子代表不同的数字，有如图所示的算式，设计一个算法求这些棋子各代表哪些数字。

$$\begin{array}{r} \text{兵炮马卒} \\ + \quad \text{兵炮车卒} \\ \hline \text{车卒马兵卒} \end{array}$$



第一行：兵 $\times 1000$ +炮 $\times 100$ +马 $\times 10$ +卒

第二行：兵 $\times 1000$ +炮 $\times 100$ +车 $\times 10$ +卒

第三行：车 $\times 10000$ +卒 $\times 1000$ +马 $\times 100$ +兵 $\times 10$ +卒

第一行 + 第二行 = 第三行

8/122

解

- 设兵、炮、马、卒和车的枚举变量分别为 a 、 b 、 c 、 d 、 e ，则 a 、 b 、 c 、 d 、 e 的取值范围均为 $0 \sim 9$ 且均不相等。
- 枚举顺序为顺序列举。

- 用 m 、 n 和 s 分别表示象棋算式中三行对应的整数，则

$$m = a \times 1000 + b \times 100 + c \times 10 + d$$

$$n = a \times 1000 + b \times 100 + e \times 10 + d$$

$$s = e \times 10000 + d \times 1000 + c \times 100 + a \times 10 + d$$

根据题目的约束条件有 $m+n=s$ 成立


```

void chess() {                                     //求象棋算式问题
    for (int a=1;a<=9;a++) {
        for (int b=0;b<=9;b++) {
            for (int c=0;c<=9;c++) {
                for (int d=0;d<=9;d++) {
                    for (int e=0;e<=9;e++) {
                        if (a==b || a==c || a==d || a==e || b==c
                            || b==d || b==e || c==d || c==e || d==e)
                            continue;              //避免重复
                        int m=a*1000+b*100+c*10+d;
                        int n=a*1000+b*100+e*10+d;
                        int s=e*10000+d*1000+c*100+a*10+d;
                        if (m+n==s)
                            printf("兵:%d 炮:%d 马:%d 卒:%d 车:%d\n",
                                a,b,c,d,e);
                    }
                }
            }
        }
    }
}

```

答案：兵:5 炮:2 马:4 卒:0 车:1

10/122

穷举算法（低效）



算法优化

穷举算法（高效）

- 选择高效的数据结构。
- 减少重复计算。
- 将原问题化为更小的问题。
- 根据问题的性质进行剪支。
- 引进其他算法。

11/122

单选题 1分

一个行李箱其密码锁为3位，共有几种密码？

- A 3
- B 100
- C 999
- D 1000**



12/91

提纲
CONTENTS

第3章—穷举法

3.1 穷举法概述

3.2 算法优化中常用的数据结构

3.3 回文串个数

3.4 最大连续子序列和

3.5 求幂集

3.6 0/1背包问题

3.7 求全排列

3.8 n皇后问题

3.9 任务分配问题

3.10 旅行商问题

13/122

3.2 算法优化中常用的数据结构

3.2.1 前缀和数组

假设有一个数组 a ，其前缀和数组用 $psum$ 表示，其中 $psum[i]$ ($0 \leq i \leq n$) 为 a 中前 i 个元素和，即 $psum[i] = a[0] + \dots + a[i-1]$ 。求 $psum$ 数组的递推关系如下：

$$psum[i] = 0$$

当 $i=0$ 时

$$psum[i] = psum[i-1] + a[i-1]$$

当 $i \geq 1$ 时

$O(n)$ 时间求出 $psum$

14/122

$$\text{psum}[i] = a[0] + \cdots + a[i-1]$$

$$\text{psum}[j+1] = a[0] + \cdots + a[i-1] + a[i] + \cdots + a[j] \quad 1 \leq i \leq j \leq n$$

两式相减：

$$\text{psum}[j+1] - \text{psum}[i] = a[i] + \cdots + a[j]$$

左右交换（满足交换律）：

$$a[i] + \cdots + a[j] = \text{psum}[j+1] - \text{psum}[i]$$



即： $a[i..j]$ 的元素和等于 $\text{psum}[j+1] - \text{psum}[i]$
($1 \leq i \leq j \leq n$)

显然 $a[i..j]$ 区间下标满足 $0 \leq i \leq j \leq n-1$ ，通过枚举 i 和 j 可以求出 a 中任意子数组元素和，用 ans 数组表示。

```
vector<vector<int>> subarr(vector<int>& a) { //求子数组和
    int n=a.size();
    vector<vector<int>> ans(n,vector<int>(n,0));
    int psum[n+1];
    psum[0]=0; //求psum前缀
    for(int i=1;i<=n;i++)
        psum[i]=psum[i-1]+a[i-1];

    for(int i=0;i<n;i++) {
        for(int j=i;j<n;j++) {
            ans[i][j]=psum[j+1]-psum[i]; //求nums[i..j]之和
        }
    }
    return ans;
}
```

16/122



【例3.2】子数组之和（LintCode138★）。给定一个整数数组 `nums`，设计一个算法找到和为0的子数组，返回满足要求的子数组的起始位置和结束位置，测试数据保证至少有一个子数组的和为0，如果有多个子数组的和为0，返回其中任意一个子数组即可。

例如，`nums={-3, 1, 2, -3, 4}`，答案为`{0, 2}`或`{1, 3}`。
要求设计如下成员函数：

```
vector<int> subarraySum(vector<int> &nums) { }
```


解法1

- 用 `vector<int>` 容器 `ans` 存放答案即和为 0 的子数组的起始位置和结束位置。
- 采用穷举法，用 `i` 和 `j` 枚举所有的 `nums[i..j]`，求出元素和 `sum`，若 `sum=0` 则置 `ans={i, j}`，返回 `ans`。

-3	1	2	-3	4
----	---	---	----	---

```

class Solution {
public:
    vector<int> subarraySum(vector<int> &nums) {
        vector<int> ans;
        int n=nums.size();
        for(int i=0;i<n;i++) {
            for(int j=i;j<n;j++) {
                int sum=0;
                for(int k=i;k<=j;k++) sum+=nums[k]; //从i到
j求累加和
                if(sum==0) {
                    ans.push_back(i); ans.push_back(j);
                    return ans; //如果和为0，结束程序
                }
            }
        }
        return {0,0}; //没有找到
    }
};

```

$O(n^3)$



超时

19/122

主观题 6分

尝试一种密码需要6秒，解开此密码锁，最坏情况下需要几分钟？



20/91

解法2

- 采用前缀和数组 **psum** 实现，设 **psum[i]** 为 **nums** 中前 **i** 个元素即 **nums[0..i-1]** 的元素和。
- 在求出 **psum** 数组后，**nums[i..j]** 的元素和等于 **psum[j+1]-psum[i]**。
- 通过枚举 **i** 和 **j** 求出元素和为 0 的 **nums[i..j]** 即可。

	0	1	2	3	4
nums	-3	1	2	-3	4
psum	0	-3	-2	0	-3

$$\text{nums}[i..j] = \text{psum}[j+1] - \text{psum}[i]$$

21/122


```

class Solution {
public:
    vector<int> subarraySum(vector<int> &nums) {
        int n=nums.size(); int psum[n+1];
        psum[0]=0;
        for(int i=1;i<=n;i++) psum[i]=psum[i-1]+nums[i-1];
        vector<int> ans;
        for(int i=0;i<n;i++) {
            for(int j=i;j<n;j++) {
                int sum=psum[j+1]-psum[i]; //求nums[i..j]之和
                if(sum==0) {
                    ans.push_back(i); ans.push_back(j);
                    return ans;
                }
            }
        }
        return {0,0};
    }
};

```

$O(n^2)$

EXE

执行用时为**904ms**，内存消耗为**10.47MB**。

22/122

上例的时间复杂度是？

- ☐ A $O(1)$
- ☐ B $O(n)$
- ☐ C $O(n*2)$
- ☒ D $O(n^2)$

```

class Solution {
public:
    vector<int> subarraySum(vector<int> &nums) {
        int n=nums.size(); int psum[n+1];
        psum[0]=0;
        for(int i=1;i<=n;i++) psum[i]=psum[i-1]+nums[i-1];
        vector<int> ans;
        for(int i=0;i<n;i++) {
            for(int j=i;j<n;j++) {
                int sum=psum[j+1]-psum[i]; //求nums[i..j]之和
                if(sum==0) {
                    ans.push_back(i); ans.push_back(j);
                    return ans;
                }
            }
        }
        return {0,0};
    }
};

```

$O(n^2)$

EXE

执行用时为**904ms**，内存消耗为**10.47MB**。

24/122

解法3

- 采用前缀和数组 **psum**+哈希表 **hmap** 进一步提高时间性能。

- 将 **psum[i]** 改为 **nums[0..i]** 共 **i+1** 个元素和即

$$\text{psum}[i] = \text{nums}[0] + \cdots + \text{nums}[i-1] + \text{nums}[i]$$

求 **psum**:

$$\text{psum}[0] = \text{nums}[0]$$

$$\text{psum}[i] = \text{psum}[i-1] + \text{nums}[i] \quad \text{当 } i > 0 \text{ 时}$$

- 对于 $j \geq i$ (即 $0 \leq i \leq j < n$) , 有

$$\text{psum}[i] = \text{nums}[0] + \cdots + \text{nums}[i],$$

$$\text{psum}[j] = \text{nums}[0] + \cdots + \text{nums}[i] + \text{nums}[i+1] + \cdots + \text{nums}[j].$$

- 两式相减得到: $\text{psum}[j] - \text{psum}[i] = \text{nums}[i+1] + \cdots + \text{nums}[j]$

- 或者 $\text{nums}[i+1] + \cdots + \text{nums}[j] = \text{psum}[j] - \text{psum}[i]$ 。

25/122

主观题 10分

 问题：

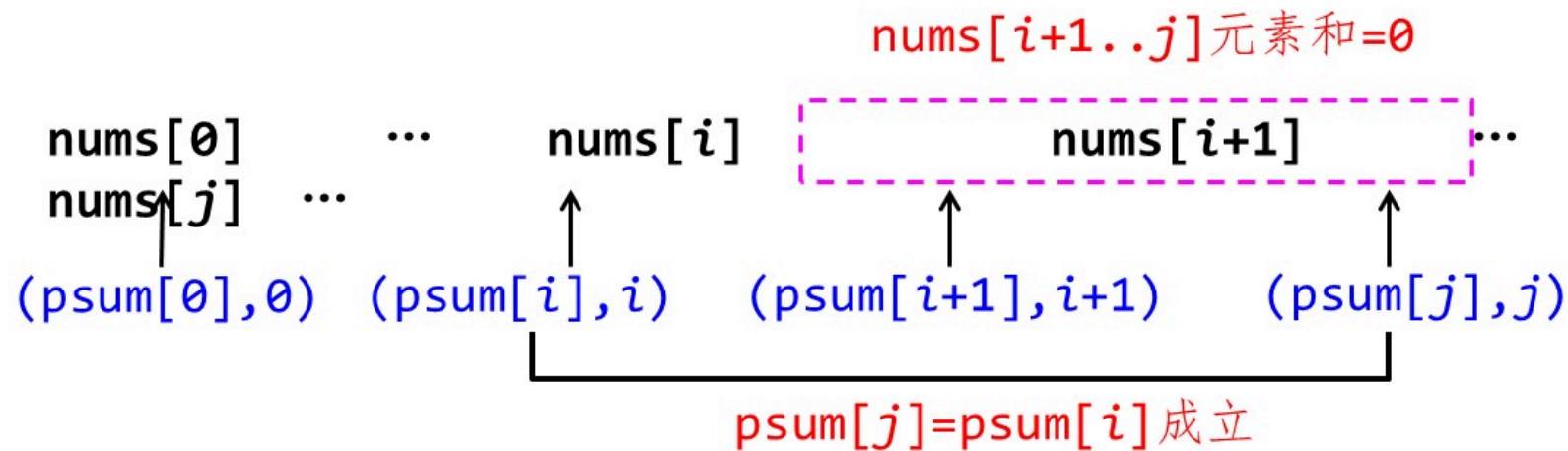
有没有更好的算法，提高上一题目的效率？



请写出优化的求解思路。

26/91

设计一个哈希表`unordered_map<int,int>`类型的容器`hmap`,
用于存放以每个`psum[i]`为关键字的序号`i`即`(psum[i], i)`



↓ 推导过程

$$\text{psum}[j] - \text{psum}[i] = \text{nums}[i+1] + \dots + \text{nums}[j] = 0$$

⇒ $\text{nums}[i+1..j]$ 的元素和为0

⇒ 答案为 $(i+1, j)$ 即 $(\text{hmap}[\text{psum}[i]]+1, j)$ 或 $(\text{hmap}[\text{psum}[j]]+1, j)$ 。

27/122

注意两点

nums[0] ... nums[i] ... nums[i+1] ...
nums[j] ...
↑ ↑ ↑ ↑
(psum[0], 0) (psum[i], i) (psum[i+1], i+1) (psum[j], j)

■ 答案起始位置为0是一种特殊情况。例如 $\text{nums} = \{1, -1, 2\}$ 。

0
1
↑
psum[0]=1
插入(1, 0)

1
-1
↑
psum[1]=0
插入(0, 1)

2
2

为了保证找到这样的答案需要事先在hmap中插入元素(0, -1)，即 $\text{hmap}[0] = -1$ 。

hmap	key	value
	0	-1
	1	0
	2	2

hmap: (0, -1), (1, 0), 找到答案{0, 1}

28/122

nums[0]	...	nums[i]		nums[i+1]	...
nums[j]	...				
↑		↑		↑	↑
(psum[0],0)	(psum[i],i)	(psum[i+1],i+1)		(psum[j],j)	

■ 不必采用前缀和数组，将**psum**改为单个变量即可。


```

class Solution {
public:
    vector<int> subarraySum(vector<int> &nums) {
        unordered_map<int,int> hmap;
        vector<int> ans;  int n=nums.size();
        hmap[0]=-1; int psum=0;
        for (int j=0;j<n;j++) {
            psum+=nums[j];
            if (hmap.count(psum)>0) {    //找到了psum
                ans.push_back(hmap[psum]+1);
            }
            ans.push_back(j);
            hmap[psum]=j;
        }
        return ans;
    }
};

```

$O(n)$

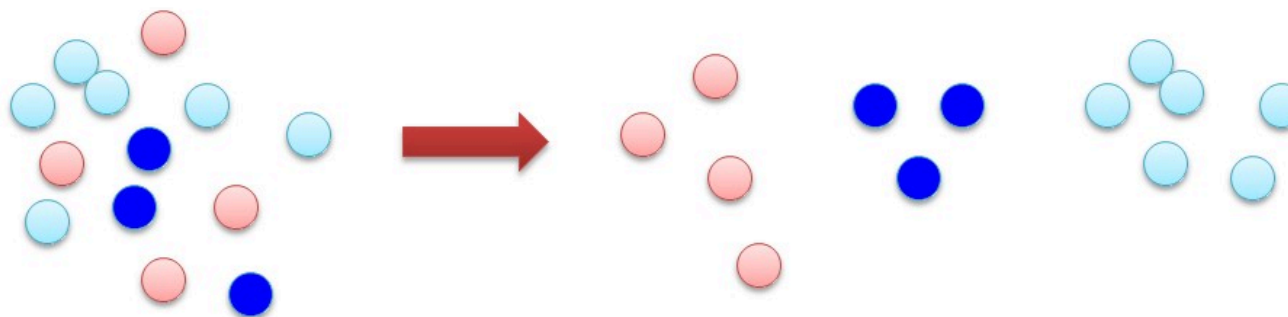


执行用时为806ms，内存消耗为10.49MB。

30/122

3.2.2 并查集

■ 并查集是解决不相交集合的一种数据结构。



1. 什么是并查集

- 给定 n 个结点的集合 U ，结点编号为 $1 \sim n$ ，再给定一个等价关系 R 。
- 由等价关系产生所有结点的一个划分，每个结点属于一个等价类，所有等价类是不相交的。
- 现在的问题是求一个结点所属的等价类，以及合并两个等价类。



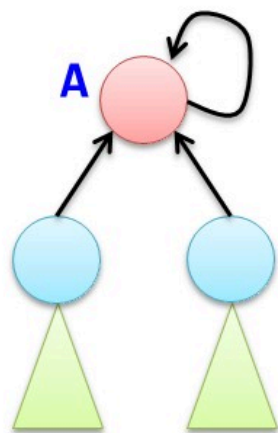
- **Init():** 初始化。
- **Find(x):** 查找 x ($x \in U$) 结点所属的等价类。
- **Union(x, y):** 将 x 和 y ($x \in U, y \in U$) 所属的两个等价类合并。

并查集

32/122

2.并查集的实现

- 采用树结构来实现。
- 将并查集看成一个森林，每个等价类用一棵树表示，包含该等价类的所有结点即结点子集，称为子集树，每个子集树通过根结点标识。



- 并查集的基本存储结构（实际上是森林的双亲存储结构）如下：

```
int parent[MAXN];      //并查集存储结构
```

```
int rnk[MAXN];         //存储结点的秩(近似于高度)
```

- $\text{parent}[i]=j$ 时，表示结点 i 的双亲结点是 j 。 $\text{rnk}[i]$ 表示子树高度即秩，秩并不与高度完全相同，但它与高度成正比。

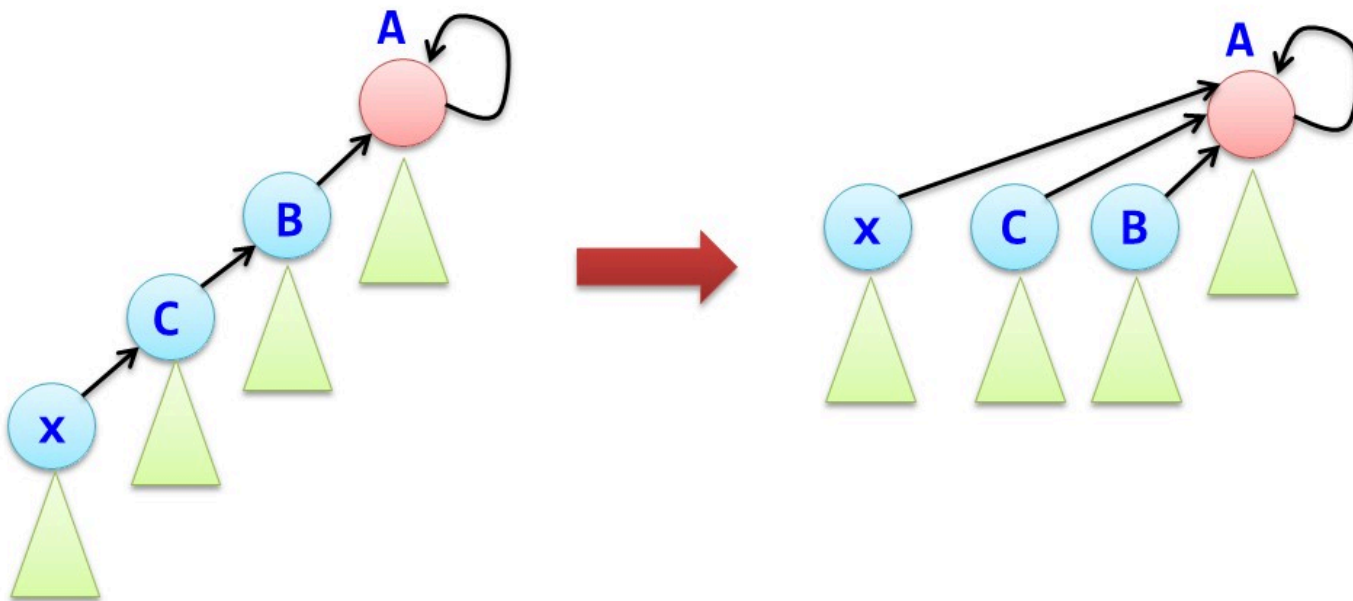
初始化算法

```
void Init() {                                     //并查集初始化
    for (int i=1;i<=n;i++) {
        parent[i]=i;
        rnk[i]=0;
    }
}
```

35/122

查找算法

- 路径查找：查找 x 结点所属子集树的根结点。
- 查找中的路径压缩：



36/122

路径查找算法（递归）：

```
int Find(int x) {           //递归算法：并查集中查找x结点的根结点
    if (x!=parent[x])
        parent[x]=Find(parent[x]);    //路径压缩
    return parent[x];
}
```

37/122

路径查找算法（非递归）：

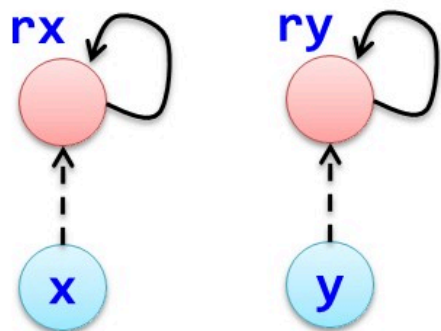
```
int Find(int x) {           //非递归算法：并查集中查找x结点的根结点
    int rx=x;
    while (parent[rx]!=rx)   //找到x的根rx
        rx=parent[rx];

    int y=x;
    while (y!=rx) {         //路径压缩
        int tmp=parent[y];
        parent[y]=rx;
        y=tmp;
    }
    return rx;              //返回根
}
```

38/122

合并算法

- 合并是给定一个等价关系 (x, y) 后，需要将 x 和 y 所属的两棵子集树合并为一棵树。
- 先查找 x 和 y 所属子集树的根结点 rx 和 ry ，若 $rx=ry$ ，说明它们同属一棵子集树，不需要合并；否则需要合并，注意合并是根结点 rx 和 ry 的合并，并且希望合并后的树高度尽可能小。



- 若 $rnk[rx] < rnk[ry]$ ，将高度较小的 rx 结点作为 ry 的孩子结点， ry 树高度不变。
- 若 $rnk[rx] > rnk[ry]$ ，将高度较小的 ry 结点作为 rx 的孩子结点， rx 树高度不变。
- 若 $rnk[rx] = rnk[ry]$ ，将 rx 结点作为 ry 的孩子结点或者 ry 结点作为 rx 的孩子结点均可，但此时合并后的树高度增1。

合并算法:

```
void Union(int x,int y) {  
    int rx=Find(x);  
    int ry=Find(y);  
    if (rx==ry) return;  
    if (rnk[rx]<rnk[ry])  
        parent[rx]=ry;  
    else {  
        if (rnk[rx]==rnk[ry])  
            rnk[rx]++;  
        parent[ry]=rx;  
    }  
}
```

//并查集中x和y的两个集合的合并

//x和y属于同一棵树的情况

//rx结点作为ry的孩子

//秩相同，合并后rx的秩增1

//ry结点作为rx的孩子

41/122



【例3.3】寻找朋友圈数（LintCode1857★★）。班上有 n （ $1 \leq n \leq 200$ ）名学生，其中有些人是朋友，有些则不是。学生的友谊具有传递性，如果已知A是B的朋友，B是C的朋友，那么A也是C的朋友。朋友圈是指所有朋友的集合。给定一个 $n \times n$ 的对称矩阵M，表示班级中学生之间的朋友关系，如果 $M[i][j]=1$ ，表示已知学生i和学生j互为朋友关系，否则为不知道。设计一个算法求所有学生中的已知的朋友圈总数。

例如， $n=3$ ， $M=\{\{1, 1, 0\}, \{1, 1, 0\}, \{0, 0, 1\}\}$ ，答案是2，两个朋友圈是 $\{0, 1\}$ 和 $\{2\}$ 。

要求设计如下成员函数：

```
int findCircleNum(vector<vector<int>> &M) { }
```

42/122

解法1

- n 名学生的编号是 $0 \sim n-1$ 。用 **fno** 数组表示顶点所在的连通分量编号。
- 首先将 n 个顶点看成 n 个连通分量，初始化顶点 i 的连通分量编号为 i 。
- 再遍历 M ，若 $M[i][j]=1$ ，在顶点 i 和顶点 j 之间连一条边，同时将顶点 j 所在连通分量的全部顶点的连通分量编号改为 **fno[i]**。
- 遍历完毕，**fno** 中不同连通分量编号的个数即为朋友圈总数。


```

class Solution {
public:
    int findCircleNum(vector<vector<int>> &M) {
        int n=M.size();
        int fno[n];    //存放每个顶点的连通分量编号
        for(int i=0;i<n;i++) fno[i]=i;
        for(int i=0;i<n;i++) {
            for(int j=i+1;j<n;j++) {
                if(M[i][j]==1) {
                    if(fno[i]==fno[j]) continue;

                    int sn1=fno[i];
                    int sn2=fno[j];
                    for(int k=0;k<n;k++) {
                        if(fno[k]==sn1) fno[k]=sn2;
                    }
                }
            }
        }
    }
}

```

44/122

```
set<int> myset;           //用set容器除重
for(int i=0;i<n;i++) {
    myset.insert(fno[i]);
}
return myset.size();
}
};
```

$O(n^3)$



执行用时为**163ms**，内存消耗为**5.21MB**。

45/122

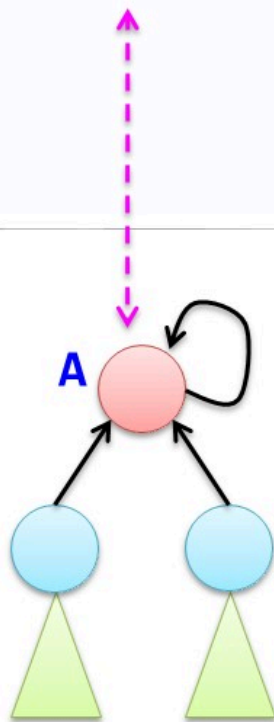
解法2

- 朋友关系是一种等价关系，求按朋友关系划分后得到的等价类的个数，为此采用并查集，每棵子集树表示一个朋友圈。
- 遍历 M ，若 $M[i][j]=1$ ，将顶点 i 和顶点 j 所在子集树合并。
- 遍历完毕，每棵子集树的根结点 i 满足 $\text{parent}[i]=i$ ，求根结点的个数即为朋友圈总数。

```
class Solution {
    int n;
    vector<int> parent;
    vector<int> rnk;
public:
    int findCircleNum(vector<vector<int>> &M) {
        n=M.size();
        parent=vector<int>(n);
        rnk=vector<int>(n);
        Init();
        for(int i=0;i<n;i++) {
            for(int j=i+1;j<n;j++) {
                if(M[i][j]==1) Union(i,j);
            }
        }
    }
};
```

47/122

```
int ans=0;  
for(int i=0;i<n;i++) {  
    if(parent[i]==i) ans++;  
}  
return ans;  
}
```



$O(n^2)$

EXE

执行用时为**142ms**，内存消耗为**4.61MB**。

48/122

提纲
CONTENTS

第3章—穷举法

3.1 穷举法概述

3.2 算法优化中常用的数据结构

3.3 回文串个数

3.4 最大连续子序列和

3.5 求幂集

3.6 0/1背包问题

3.7 求全排列

3.8 n皇后问题

3.9 任务分配问题

3.10 旅行商问题

49/122



3.3 回文串个数

问题描述：回文子串（**LintCode1856★**）。小明喜欢玩文字游戏，今天他希望在一个字符串的子串中找到回文串。回文串是从左往右和从右往左读相同的字符串，例如"**121**"和"**tacocat**"是回文串。子串是一个字符串中任意几个连续的字符构成的字符串。现在给定一个字符串**s**，设计一个算法求出**s**的回文串个数。

例如，**s="mokkori"**，它的一些子串是"**m**"，"**o**"，"**k**"，"**r**"，"**i**"，"**mo**"，"**ok**"，"**mok**"，"**okk**"，"**kk**"，"**okko**"，其中"**m**"，"**o**"，"**k**"，"**r**"，"**i**"，"**kk**"和"**okko**"是回文子串，总共有**7**个不同的回文子串，答案是**7**。

要求设计如下成员函数：

```
int countSubstrings(string &s) { }
```

50/122

 **问题描述：**回文子串（LintCode1856★）。

小明喜欢玩文字游戏，今天他希望在一个字符串的子串中找到回文串。回文串是从左往右和从右往左读相同的字符串，例如"**121**"和"**tacocat**"是回文串。子串是一个字符串中任意几个连续的字符构成的字符串。现在给定一个字符串**s**，设计一个算法求出**s**的回文串个数。

现有，**s="mokkori"**，列出它的所有子串：

解法1

- 这里是求 **s** 中不重复的回文子串个数，由于 **s** 中存在重复的字符，所以可能存在两个位置不同的子串。
- 为了到达子串除重目的，设计一个 **set<string>** 容器 **myset**。
- 枚举所有 **s[i..j]** ($0 \leq i \leq j \leq n-1$)，若 **s[i..j]** 是回文，将其插入到 **myset** 中，最后返回 **myset** 的大小。

主观题 10分

 **问题描述：**回文子串（LintCode1856★）。小明喜欢玩文字游戏，今天他希望在一个字符串的子串中找到回文串。回文串是从左往右和从右往左读相同的字符串，例如"121"和"tacocat"是回文串。子串是一个字符串中任意几个连续的字符构成的字符串。现在给定一个字符串s，设计一个算法求出s的回文串个数。

例如，s="mokkori"，它的一些子串是"m"，"o"，"k"，"r"，"i"，"mo"，"ok"，"mok"，"okk"，"kk"，"okko"，其中"m"，"o"，"k"，"r"，"i"，"kk"和"okko"是回文子串，总共有7个不同的回文子串，答案是7。

要求设计如下成员函数：



```
int countSubstrings(string &s) { }
```

写写用穷举法求解的思路。

52/91


```
class Solution {
public:
    int countSubstrings(string &s) {
        int n=s.size();
        set<string> myset;
        for(int i=0;i<n;i++) {
            for(int j=i;j<n;j++) {
                if(ispal(s,i,j))    //s[i..j]为回文
                    myset.insert(s.substr(i,j-i+1));
            }
        }
        return myset.size();
    }
}
```

53/122

```
bool ispal(string &s,int low,int high) {  
    //判断s[i..j]是否为回文  
    int i=low,j=high;  
    while(i<j) {  
        if(s[i] != s[j]) return false;  
        i++; j--;  
    }  
    return true;  
}  
};
```



执行用时为**507ms**，内存消耗为**2.07MB**。

54/122

解法2

- 上述算法在回文判断中存在冗余，例如 $s = \text{"abba"}$ ，其中 **"bb"** 和 **"abba"** 的回文判断是独立的，实际上当确定 **"bb"** 为回文后，左右两边均为 'a'，则可以得出 **"abba"** 也是回文，以此作为优化点提高算法性能。
- 两种回文子串的中心点。

abba abba

↑ ↑

```

class Solution {
    int n;
    set<string> myset;
public:
    int countSubstrings(string &s) {
        n=s.size();
        for(int c=0;c<n;c++)           //考虑每个字符为回文中心点
            cnt(s,c,c);

        for(int c=0;c<n-1;c++)        //考虑每两个字符中间为回文中心
            cnt(s,c,c+1);
        return myset.size();
    }
}

```

abba
↑

abba
↑


```
void cnt(string &s,int l,int r) {           //求回文子串
    while (l>=0 && r<n && s[l]==s[r]) {
        myset.insert(s.substr(l,r-l+1));
        l--; r++;
    }
};
```



执行用时为**122ms**，内存消耗为**5.45MB**。

57/122

提纲
CONTENTS

第3章—穷举法

3.1 穷举法概述

3.2 算法优化中常用的数据结构

3.3 回文串个数

3.4 最大连续子序列和

3.5 求幂集

3.6 0/1背包问题

3.7 求全排列

3.8 n皇后问题

3.9 任务分配问题

3.10 旅行商问题

58/122

3.4 最大连续子序列和

 **问题描述：** 给定一个含 n ($n \geq 1$) 个整数的序列，要求求出其中最大连续子序列的和。

序列 $(-2, \underline{11, -4, 13}, -5, -2)$ 的最大连续子序列和为 **20**。

序列 $(-6, 2, 4, -7, \underline{5, 3, 2, -1, 6, -9, 10}, -2)$ 的最大连续子序列和为 **16**。

规定一个序列最大连续子序列和至少是**0**，如果小于**0**，其结果为**0**。

解法1

n 个整数的序列 $a[0..n-1]$ ，枚举所有连续子序列 $a[i..j]$ 。

$a[0..5]=\{-2, 11, -4, 13, -5, -2\}$

	$j \downarrow$	0	1	2	3	4	5	
		-2	11	-4	13	-5	-2	初始序列
$i \rightarrow$	0	-2	9	5	18	13	11	
	1		11	7	20	15	13	
	2			-4	9	4	2	
	3				13	8	6	
	4					-5	-7	
	5						-2	

60/122

```

int maxsubsum1(int a[],int n) {           //解法1
    int maxsum=0,cursum;
    for (int i=0;i<n;i++) {
        for (int j=i;j<n;j++) {
            cursum=0;
            for (int k=i;k<=j;k++)
                cursum+=a[k];
            maxsum=max(maxsum,cursum);    //比较求最大maxsum
        }
    }
    return maxsum;
}

```

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} \sum_{k=i}^j 1 = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} (j-i+1) = \frac{1}{2} \sum_{i=0}^{n-1} (n-i)(n-i+1) = O(n^3)$$

解法2

- 采用前缀和方法，用 $\text{psum}(i)$ 表示子序列 $a[0..i-1]$ 元素和即 a 中前 i 个元素和，显然有如下递推关系：

$$\text{psum}(0)=0$$

$$\text{psum}(i)=\text{psum}(i-1)+a[i-1] \quad \text{当 } i>0 \text{ 时}$$

- 假设 $j \geq i$ ，则有：

$$\text{psum}[i]=a[0]+a[1]+\cdots+a[i-1]$$

$$\text{psum}[j]=a[0]+a[1]+\cdots+a[i-1]+a[i]+\cdots+a[j-1]$$

- 两式相减得的 $\text{psum}[j]-\text{psum}[i]=a[i]+\cdots+a[j-1]$ 。
- i 从 0 到 $n-1$ 、 $j-1$ 从 i 到 $n-1$ （即 j 从 $i+1$ 到 n ）循环可以求出所有连续子序列 $a[i..j]$ 之和，比较求最大值 maxsum 。

```

int maxsubsum2(int a[],int n) {           //解法2
    int psum[n+1];
    psum[0]=0;
    for(int i=1;i<=n;i++)
        psum[i]=psum[i-1]+a[i-1];

    int maxsum=0,cursum;
    for (int i=0;i<n;i++) {
        for (int j=i+1;j<=n;j++) {
            cursum=psum[j]-psum[i];
            maxsum=max(maxsum,cursum);    //比较求最大maxsum
        }
    }
    return maxsum;
}

```

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i+1}^n 1 = \sum_{i=0}^{n-1} (n-i) = \frac{n(n+1)}{2} = O(n^2)$$

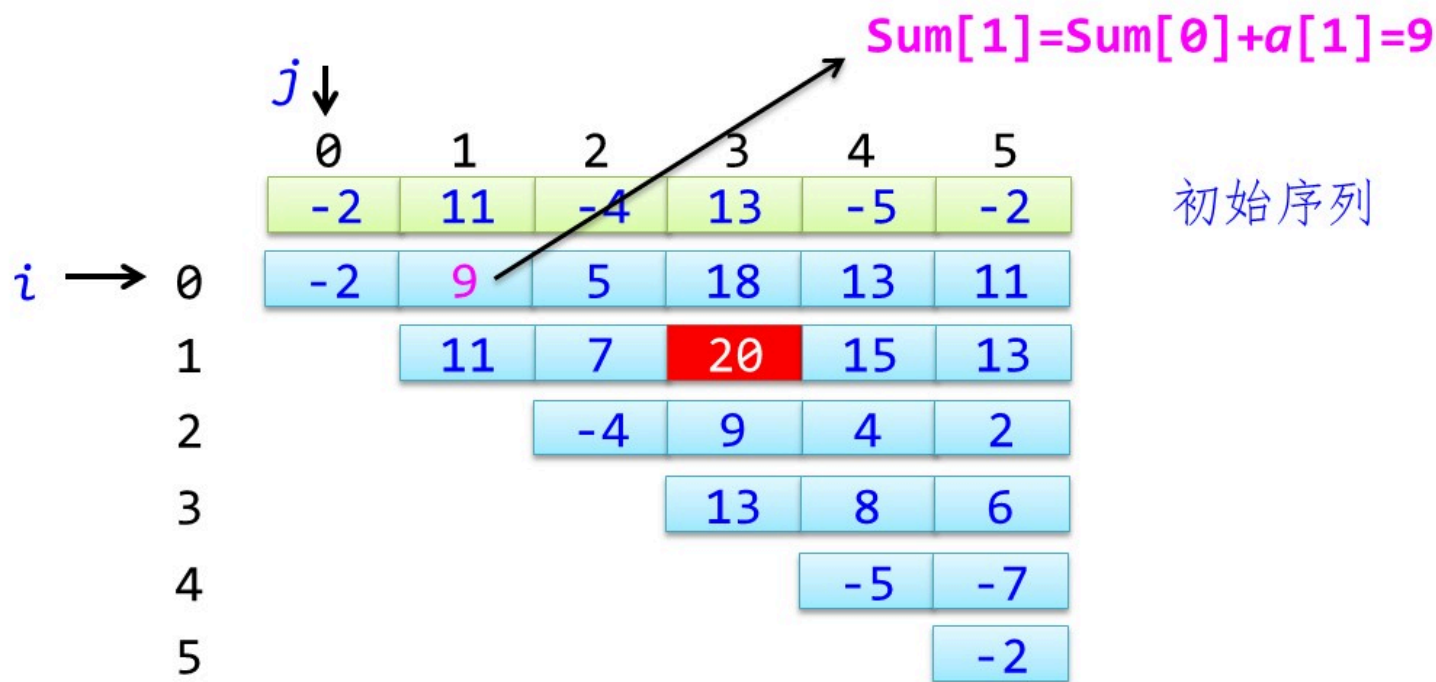
63/122

解法3 优化点：避免起始下标*i*开始的子序列的重复计算。

- 用 $\text{Sum}(a[i..j])$ 表示子序列 $a[i..j]$ 元素和，初始时置 $\text{Sum}(a[i..j])=0$ ，显然有如下递推关系：

$$\text{Sum}(a[i..j]) = \text{Sum}(a[i..j-1]) + a[j] \quad \text{当 } j \geq i \text{ 时}$$

- 连续求 $a[i..j]$ 子序列和（ $j=i, i+1, \dots, n-1$ ）时没有
必要使用循环变量为*k*的第3重循环。



64/122

```

int maxsubsum3(int a[],int n) {           //解法3
    int maxsum=0,cursum;
    for (int i=0;i<n;i++) {
        cursum=0;
        for (int j=i;j<n;j++) {
            cursum+=a[j];
            maxsum=max(maxsum,cursum);    //比较求最大maxsum
        }
    }
    return maxsum;
}

```

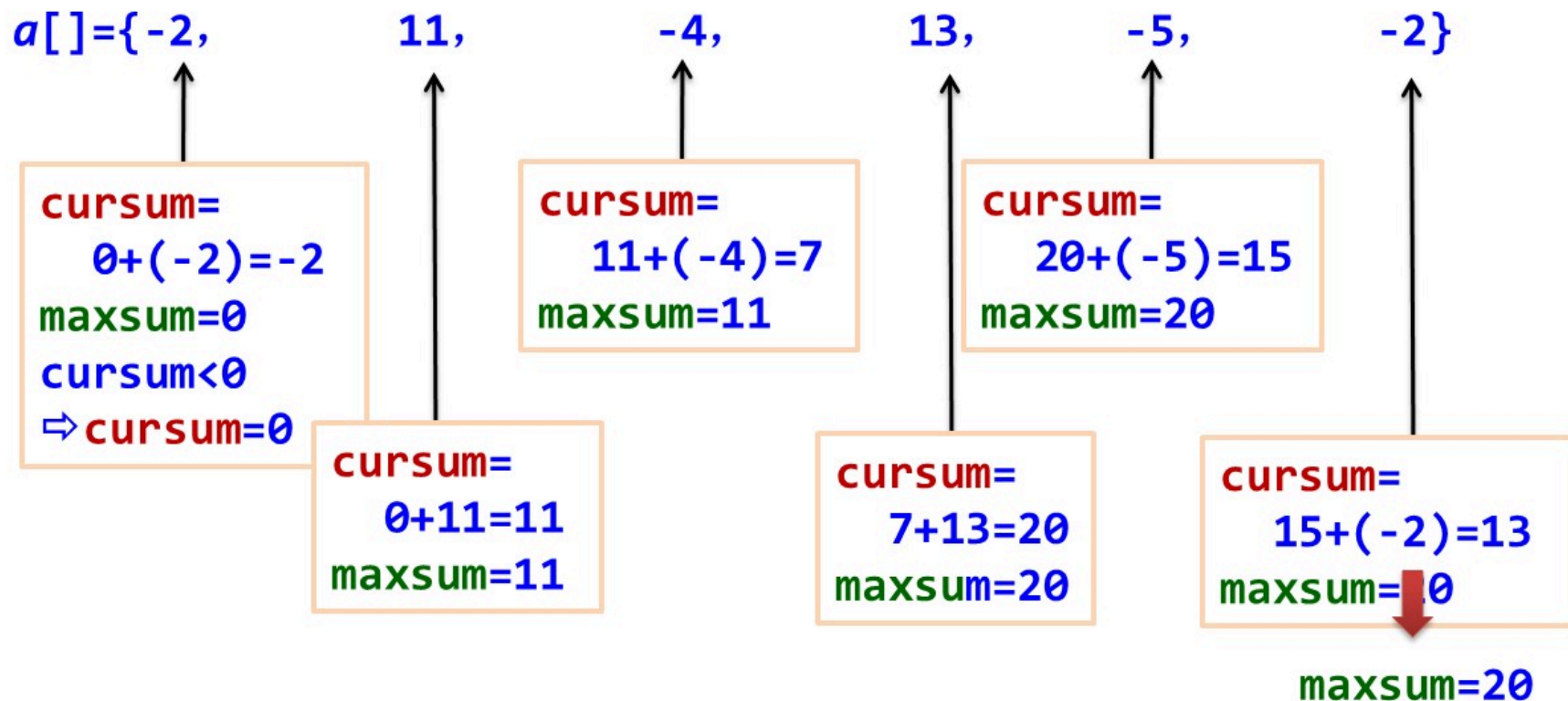
$$T(n) = \sum_{i=0}^{n-1} \sum_{j=i}^{n-1} 1 = \sum_{i=0}^{n-1} (n-i) = \frac{n(n+1)}{2} = O(n^2)$$

65/122

解法4

优化点: maxsum 至少为0。

$\text{maxsum}=0, \text{cursum}=0$ (cursum 表示以 $a[i]$ 结尾的最大和)



66/122


```

int maxsubsum4(int a[],int n) {           //解法4
    int maxsum=0,cursum=0;
    for (int i=0;i<n;i++) {
        cursum+=a[i];
        if(cursum>=maxsum)                //比较求最大maxsum
            maxsum=cursum;
        if(cursum<0)                      //若cursum<0, 从下一个位置开始
            cursum=0;
    }
    return max(maxsum,0);
}

```

$$T(n)=O(n)$$

67/122



【例3.4】最大子序和（LeetCode53★）。给定一个含 n （ $1 \leq n \leq 10^5$ ）个整数的数组 `nums`，设计一个算法找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

例如，`nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4}`，答案为6。

要求设计如下成员函数：

```
int maxSubArray(vector<int>&nums) { }
```

解

■ 最大连续子序列和可能为负数，例如 $\text{nums}=\{-1, -2\}$ ，结果为 -1 ，因此不能简单采用3.1.2节解法4的优化点，需要做两点修改：

(1) maxsum 初始化为 $\text{nums}[0]$ 而不是 0 。

(2) 在求出 maxsum 后直接返回 maxsum ，而不是 $\text{max}(\text{maxsum}, 0)$ 。

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int n=nums.size();
        if(n==1) return nums[0];
        int maxsum=nums[0],cursum=0;
        for (int i=0;i<n;i++) {
            cursum+=nums[i];
            if(cursum>=maxsum)           //求maxsum
                maxsum=cursum;
            if(cursum<0)                 //若cursum<0, 重置其为0
                cursum=0;
        }
        return maxsum;
    }
};
```

EXE

执行用时为**92ms**，内存消耗为**66.1MB**。

70/122

提纲
CONTENTS

第3章—穷举法

3.1 穷举法概述

3.2 算法优化中常用的数据结构

3.3 回文串个数

3.4 最大连续子序列和

3.5 求幂集**

3.6 0/1背包问题

3.7 求全排列

3.8 n皇后问题

3.9 任务分配问题

3.10 旅行商问题

71/122

3.5 求幂集

 **问题描述：**对于给定的正整数 n ($n \geq 1$)，求 $1 \sim n$ 构成的集合的幂集 ps 。

例如， $n=3$ 时 $ps=\{ \{\}, \{1\}, \{2\}, \{1, 2\}, \{3\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\} \}$ （顺序可以任意）。

解法1

二进制表示法

$n=3$

子集	对应的二进制数	对应的十进制数
$\{\}$	000	0
$\{1\}$	001	1
$\{2\}$	010	2
$\{1, 2\}$	011	3
$\{3\}$	100	4
$\{1, 3\}$	101	5
$\{2, 3\}$	110	6
$\{1, 2, 3\}$	111	7



73/8

归纳:

二进制位值	*	*	*	*	*
二进制位序号	$n-1$	...	j	...	0
					
1~ n 集合元素	n	...	$j+1$	...	1

- 若二进制位 j 是1, 则对应的子集中包含整数 $j+1$, 否则不包含整数 $j+1$ 。
- 最大二进制是 n 个1, 对应十进制数 2^n-1 。
- 0到 2^n-1 的每个十进制数表示一个子集。

<code>vector<vector<int>> pset1(int n) {</code>	<code>//求幂集算法1</code>
<code>vector<vector<int>> ps;</code>	<code>//存放幂集</code>
<code>for(int i=0;i<(1<<n);i++) {</code>	<code>//执行2^n次</code>
<code>vector<int> e;</code>	
<code>for(int j=0;j<n;j++) {</code>	
<code>if(i&(1<<j))</code>	<code>//i的二进制位j的值是1</code>
<code> e.push_back(j+1);</code>	<code>//选取整数j+1</code>
<code>}</code>	
<code>ps.push_back(e);</code>	<code>//将子集e添加到ps中</code>
<code>}</code>	
<code>return ps;</code>	
<code>}</code>	

时间复杂度: $O(n \times 2^n)$

75/122

解法2

- 用 `vector<vector<int>>` 表示幂集。
- 增量穷举法：以 $n=2$ 为例，求 $1\sim 2$ 的幂集的过程：

初始幂集 M_0 : $\{\{\}\}$

↓ M_0 中每个集合元素添加 **1** 得到 A_1

A_1 : $\{\{1\}\}$

↓ $M_1 = M_0 \cup A_1$

$\{1\}$ 的幂集 M_1 : $\{\{\}, \{1\}\}$

↓ M_1 中每个集合元素添加 **2** 得到 A_2

A_2 : $\{\{2\}, \{1, 2\}\}$

↓ $M_2 = M_1 \cup A_2$

$\{1, 2\}$ 的幂集 M_2 : $\{\{\}, \{1\}, \{2\}, \{1, 2\}\}$


```

vector<vector<int>> pset2(int n) { //求幂集算法2
    vector<vector<int>> ps;        //存放幂集
    vector<int> e;
    ps.push_back(e);              //添加{}空集合元素
    for (int i=1;i<=n;i++) {      //循环添加1~n
        vector<vector<int>> A=ps;  //A存放上一步得到的幂集
        for (auto it=A.begin();it!=A.end();++it)
            (*it).push_back(i);    //在A每个集合元素末尾添加i
        for (auto it=A.begin();it!=A.end();++it)
            ps.push_back(*it);     //将A每个集合元素添加到ps中
    }
    return ps;
}

```

77/122

不用A，直接在ps中添加A的元素

```
vector<vector<int>> pset3(int n) {  
    vector<vector<int> > ps;  
    vector<int> e;  
    ps.push_back(e);  
    for (int i=1;i<=n;i++) {  
        int m=ps.size();  
        for(int j=0;j<m;j++) {  
            vector<int> e=ps[j];  
            e.push_back(i);  
            ps.push_back(e);  
        }  
    }  
    return ps;  
}
```

//求幂集算法3
//存放幂集

//添加{}空集合元素
//循环添加1~n

//取出e=ps[j]
//在e的末尾添加i
//再将e添加到ps中

78/122

递归算法

```
vector<vector<int>> pset4(int n) {           //求幂集算法4
    if(n==0) return {{}};
    vector<vector<int>> ps;                  //存放幂集
    ps=pset4(n-1);                          //求出1-n-1的幂集ps
    int m=ps.size();
    for(int j=0;j<m;j++) {
        vector<int> e=ps[j];               //取出e=ps[j]
        e.push_back(n);                    //在e的末尾添加n
        ps.push_back(e);                  //再将e添加到ps中
    }
    return ps;
}
```

79/122



【例3.5】子集（LintCode17★★）。给定一个含不同整数的集合**nums**，设计一个算法求其所有的子集，要求子集中的元素不能以降序排列，解集不能包含重复的子集，但全部的子集可以按任意顺序输出。

例如，**nums**={2, 3, 1}，答案是{{3}, {1}, {2}, {1, 2, 3}, {1, 3}, {2, 3}, {1, 2}, {}}。

要求设计如下成员函数：

```
vector<vector<int>> subsets(vector<int> &nums) { }
```


解法1

- 采用二进制解法。
- 将求 $1 \sim n$ 集合的幂集改为求`nums[0..n-1]`集合的幂集。
- 由于要求子集中的元素不能以降序排列，先就`nums`递增排序。


```

class Solution {
public:
    vector<vector<int>> subsets(vector<int> &nums) {
        sort(nums.begin(),nums.end());
        vector<vector<int>> ps;           //存放幂集
        int n=nums.size();
        for(int i=0;i<(1<<n);i++) {      //执行2^n次
            vector<int> e;
            for(int j=0;j<n;j++) {
                if(i&(1<<j)) e.push_back(nums[j]);
            }
            ps.push_back(e);
        }
        return ps;
    }
};

```



执行用时为**41ms**，内存消耗为**3.82MB**。

82/122

解法2

- 用 `vector<vector<int>>` 表示幂集。
- 增量穷举法：以 $n=2$ 为例，求 $1\sim 2$ 的幂集的过程：

初始幂集 M_0 : $\{\{\}\}$

↓ M_0 中每个集合元素添加 **1** 得到 A_1

A_1 : $\{\{1\}\}$

↓ $M_1 = M_0 \cup A_1$

$\{1\}$ 的幂集 M_1 : $\{\{\}, \{1\}\}$

↓ M_1 中每个集合元素添加 **2** 得到 A_2

A_2 : $\{\{2\}, \{1, 2\}\}$

↓ $M_2 = M_1 \cup A_2$

$\{1, 2\}$ 的幂集 M_2 : $\{\{\}, \{1\}, \{2\}, \{1, 2\}\}$

- 将求 $1 \sim n$ 集合的幂集改为求 $\text{nums}[0..n-1]$ 集合的幂集。
- 由于要求子集中的元素不能以降序排列，先就 nums 递增排序。
- 初始化 $\text{ps}=\{\{\}\}$ ， i 从 0 到 $n-1$ 循环：置 $\text{A}=\text{ps}$ ，在 A 的每个集合元素中添加 $\text{nums}[i]$ ，再将 A 的元素添加的 ps 中。

```

class Solution {
public:
    vector<vector<int>> subsets(vector<int> &nums) {
        int n=nums.size();
        if(n==0) return {{}};
        sort(nums.begin(),nums.end());
        vector<vector<int>> ps;           //存放幂集
        vector<int> e;
        ps.push_back(e);                //添加{}空集合元素
        for (int i=0;i<n;i++) {          //循环添加nums[0..n-1]
            vector<vector<int>> A=ps;      //A存放上一步得到的幂集
            for (auto it=A.begin();it!=A.end();++it)
                (*it).push_back(nums[i]); //A每个元素尾添加nums[i]
            for (auto it=A.begin();it!=A.end();++it)
                ps.push_back(*it);        //A中集合元素添加到ps中
        }
        return ps;
    }
};

```

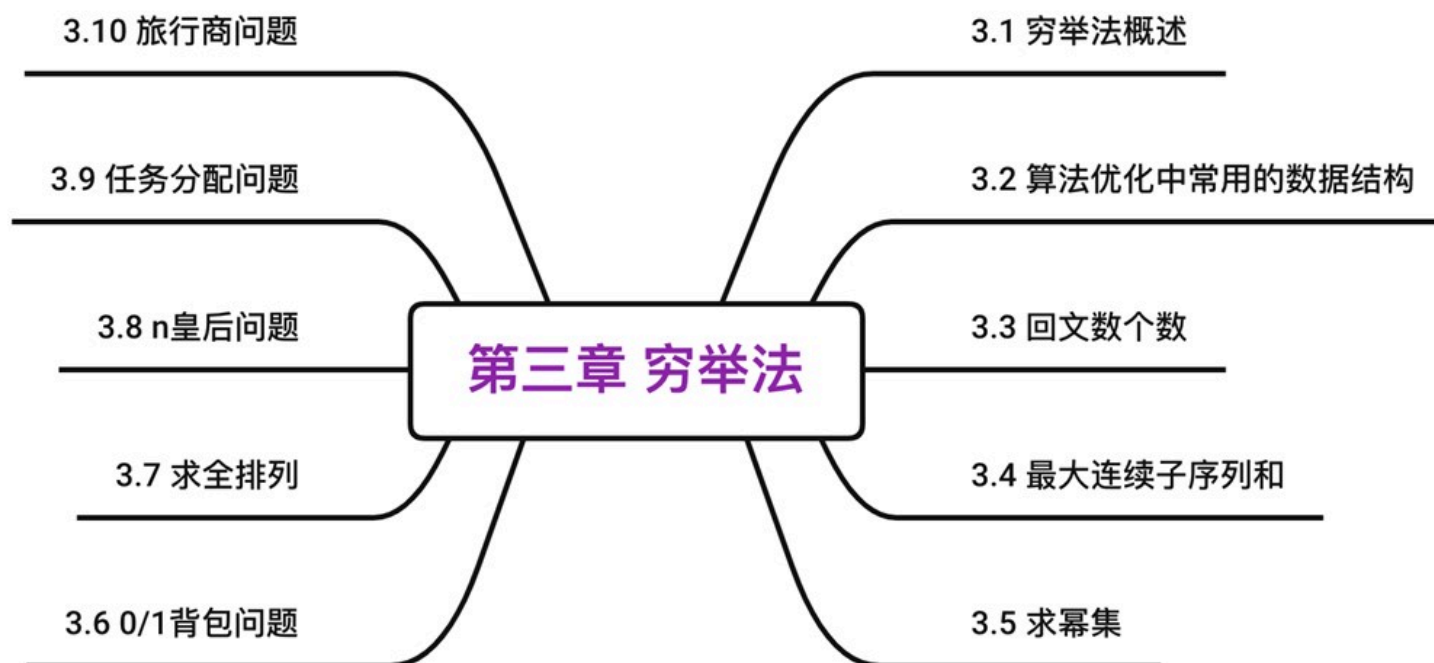
EXE

执行用时为**41ms**，内存消耗为**3.89MB**。

85/122

提纲
CONTENTS

第3章—穷举法



86/122



路虽远行则将至，事虽难做则必成。

87/14